

Testkonzept für das Open-Source-Projekt „lazygit,,

Belegarbeit im Studiengang Informatik (B.Sc.)

**Grundlagen des Softwaretestens
Wintersemester 2025/26**

Dozent: Prof. Dr. Matthias Längrich

Hochschule Zittau/Görlitz
Fakultät Elektrotechnik und Informatik

**Verfasser: Konrad Miosga
Matrikelnummer: 1056710
E-Mail-Adresse: konrad.miosga@stud.hszg.de**

Datum der Abgabe: 22.01.2026

Inhaltsverzeichnis

1. Einleitung	1
1.1. Projektkontext	1
1.2. Zielsetzung und Methodik	1
2. Theoretische Grundlagen	2
3. Analyse der vorhandenen Teststrategie	4
3.1. Unit-Tests	4
3.2. Table-Driven Tests	4
3.3. Integrationstests	5
4. Continuous Integration Pipeline	7
5. Entwurf eigener Testfälle	8
5.1. Identifikation von Testlücken	8
5.2. Analyse und Testfalldesign	8
5.3. Implementierung	9
5.4. Testergebnisse und Coverage-Verbesserung	9
6. Testauswertung und Metriken	10
7. Fazit	11
Quellen	12

1. Einleitung

1.1. Projektkontext

In dieser Belegarbeit wird das Testkonzept des Open-Source-Projekts **lazygit** analysiert, bewertet und durch eigene Testfälle ergänzt. Lazygit ist eine in Go entwickelte Terminal-UI für Git-Kommandos, die komplexe Git-Operationen durch eine intuitive, tastaturgesteuerte Benutzeroberfläche vereinfacht. Das Projekt wurde von Jesse Duffield initiiert, wird seit 2018 aktiv entwickelt und gehört mit über 50.000 GitHub-Stars zu den populärsten Git-Tools im Terminal-Bereich.

Die Architektur folgt dem MVC-Muster mit klarer Trennung zwischen UI-Komponenten, Business-Logik und Git-Operationen. Diese Struktur erleichtert das isolierte Testen einzelner Komponenten erheblich.

1.2. Zielsetzung und Methodik

Diese Arbeit verfolgt mehrere Ziele: Analyse der bestehenden Teststrategie mit ihren Stärken und Schwächen, Identifikation von Testlücken durch Coverage-Analysen und Code-Reviews, praktische Implementierung neuer Testfälle sowie Ableitung konkreter Handlungsempfehlungen.

Die Bearbeitung erfolgte systematisch: Nach Literaturrecherche zu Testing-Grundlagen wurde die bestehende Testinfrastruktur analysiert. Coverage-Reports und manuelle Code-Reviews identifizierten Testlücken, die anschließend durch table-driven Tests geschlossen wurden. Die neuen Tests wurden in die CI-Pipeline integriert und auf mehreren Plattformen validiert.

2. Theoretische Grundlagen

Software-Testing ist ein zentraler Bestandteil der Qualitätssicherung in der Softwareentwicklung. Das oft zitierte Statement von Edsger W. Dijkstra bringt dabei eine grundlegende Eigenschaft des Testens prägnant auf den Punkt: Tests eignen sich hervorragend zum Aufdecken von Fehlern, sind jedoch ungeeignet, um die vollständige Fehlerfreiheit eines Programms nachzuweisen [1].

Diese Einschränkung ergibt sich aus der Natur des Software-Testings als Stichprobenverfahren. Da Programme in der Regel eine sehr große Menge möglicher Eingaben besitzen, kann im Rahmen von Tests nur eine endliche Teilmenge davon überprüft werden. Ein formaler Beweis der Korrektheit allein durch Tests ist daher prinzipiell nicht möglich.

Vor diesem Hintergrund kommt der systematischen und zielgerichteten Auswahl repräsentativer Testfälle eine zentrale Bedeutung zu. Ziel des Testens ist es nicht, wie schon eingangs erwähnt, Fehlerfreiheit zu garantieren, sondern mit begrenztem Aufwand eine möglichst hohe Wahrscheinlichkeit zur Entdeckung relevanter Defekte zu erreichen.

Der Testprozess lässt sich grundsätzlich in zwei zentrale Bereiche gliedern: die statische Analyse und das dynamische Testen [2, S. 4]. Die statische Analyse untersucht Softwareartefakte ohne deren Ausführung und zielt darauf ab, potenzielle Fehler, Regelverletzungen oder Qualitätsmängel frühzeitig im Entwicklungsprozess zu identifizieren [2, S. 43–45]. Zu den typischen Verfahren zählen Code-Reviews, der Einsatz von Lintern sowie statische Datenfluss- und Kontrollflussanalysen.

Demgegenüber steht das dynamische Testen, bei dem die Software mit konkreten Eingabedaten ausgeführt wird. Dadurch lassen sich insbesondere solche Fehler aufdecken, die erst zur Laufzeit auftreten, etwa Speicherlecks, Race Conditions oder fehlerhaftes Laufzeitverhalten. Dynamische Tests ermöglichen somit eine Überprüfung des tatsächlichen Systemverhaltens unter realistischen oder gezielt konstruierten Bedingungen [2, S. 39–43]. Der Schwerpunkt dieser Arbeit liegt auf dem dynamischen Testen und den zugehörigen Testverfahren.

Dynamisches Testen lässt sich grundsätzlich in Black-Box-Tests und White-Box-Tests unterteilen. Black-Box-Tests leiten Testfälle ausschließlich aus Spezifikationen und Anforderungen ab, ohne Kenntnis der internen Programmstruktur. Typische Verfahren sind hierbei die Äquivalenzklassenbildung sowie darauf aufbauend die Grenzwertanalyse. White-Box-Tests hingegen konstruieren Testfälle auf Basis der internen Programmstruktur mit dem Ziel, definierte Abdeckungskriterien wie Anweisungs- oder Pfadabdeckung zu erfüllen [3, S. 173–174]. In der Testpraxis werden beide Ansätze häufig kombiniert, um sowohl funktionale als auch strukturelle Aspekte der Software zu überprüfen.

Innerhalb der White-Box-Tests lassen sich verschiedene, aufeinander aufbauende Testverfahren unterscheiden. Unit-Tests überprüfen die kleinsten testbaren Einheiten eines Programms in Isolation. Abhängige Komponenten werden dabei typischerweise durch Mocks oder Stubs ersetzt, was schnelle, reproduzierbare und deterministische Tests ermöglicht [2, S. 371–372]. Eine häufig eingesetzte Ausprägung sind sogenannte Table-Driven Tests, bei denen mehrere Testfälle in Form von Datentabellen definiert und von einem generischen Testcode iterativ ausgeführt werden [4]. Dieses Vorgehen reduziert Redundanz und verbessert die Wartbarkeit der Tests.

Aufbauend auf den Unit-Tests folgen Integrationstests, die das Zusammenwirken mehrerer bereits getesteter Module überprüfen. Ziel ist es insbesondere, Fehler an Schnittstellen sowie Probleme im Zusammenspiel der Komponenten aufzudecken [2, S. 372–376]. Im Gegensatz zu Unit-Tests kommen hierbei überwiegend reale Komponenten statt isolierender Mocks zum Einsatz, wodurch eine höhere Aussagekraft hinsichtlich der Gesamtfunktionalität des Systems erreicht wird.

3. Analyse der vorhandenen Teststrategie

Lazygit testet auf zwei Ebenen: Unit-Tests und Integrationstests. Die Unit-Tests prüfen einzelne Funktionen isoliert und laufen sehr schnell, während die Integrationstests die gesamte Anwendung mit echter Benutzerinteraktion testen. Dabei kommen sogenannte Table-Driven Tests zum Einsatz, eine Go-typische Methode, um viele ähnliche Testfälle kompakt zu schreiben.

3.1. Unit-Tests

Die Unit-Tests in Lazygit arbeiten mit Mocks, das heißt sie führen keine echten Git-Befehle aus, sondern simulieren diese. Das Kernstück ist der `FakeCmdObjRunner`, der vorgibt, Git-Befehle auszuführen, tatsächlich aber nur aufzeichnet, welche Befehle aufgerufen wurden, und vordefinierte Antworten zurückgibt. So kann man Tests schreiben, die schnell, zuverlässig und unabhängig vom tatsächlichen Git-Zustand sind.

Ein typischer Unit-Test sieht ungefähr so aus:

```
func TestBranchNewBranch(t *testing.T) {
    runner := oscommands.NewFakeRunner(t).
        ExpectGitArgs([]string{"checkout", "-b", "test", "refs/heads/master"}, "", nil)
    instance := buildBranchCommands(commonDeps{runner: runner})

    assert.NoError(t, instance.New("test", "refs/heads/master"))
    runner.CheckForMissingCalls()
}
```

Man sagt dem Fake-Runner vorher, welchen Git-Befehl man erwartet, führt dann die zu testende Funktion aus, und prüft am Ende, ob wirklich der erwartete Befehl aufgerufen wurde. Das ist einfach und funktioniert gut für Komponenten, die hauptsächlich Git-Befehle zusammenbauen und ausführen.

3.2. Table-Driven Tests

Table-Driven Tests sind in Go sehr verbreitet. Die Idee ist, dass man mehrere Testfälle in einer Liste definiert und dann in einer Schleife durchläuft. Das spart Code-Duplikation und macht es einfach, neue Testfälle hinzuzufügen. In Lazygit wird das konsequent eingesetzt, besonders wenn man verschiedene Eingaben und Fehlerfälle durchprobieren will.

Ein Beispiel aus `branch_test.go` zeigt, wie das aussieht:

```

scenarios := []scenario{
    {
        "Can't retrieve pushable count",
        oscommands.NewFakeRunner(t).
            ExpectGitArgs([]string{"rev-list", "@{u}..HEAD", "--count"}, "",
errors.New("error")),
        "?", "?",
    },
    {
        "Retrieve pullable and pushable count",
        oscommands.NewFakeRunner(t).
            ExpectGitArgs([]string{"rev-list", "@{u}..HEAD", "--count"}, "1\n", nil).
            ExpectGitArgs([]string{"rev-list", "HEAD..@{u}", "--count"}, "2\n", nil),
        "1", "2",
    },
}

for _, s := range scenarios {
    t.Run(s.testName, func(t *testing.T) {
        instance := buildBranchCommands(commonDeps{runner: s.runner})
        pushables, pullables := instance.GetCommitDifferences("HEAD", "@{u}")
        assert.EqualValues(t, s.expectedPushables, pushables)
        assert.EqualValues(t, s.expectedPullables, pullables)
        s.runner.CheckForMissingCalls()
    })
}

```

Jedes Szenario hat einen Namen, eine Mock-Konfiguration und erwartete Ergebnisse. Die Schleife führt dann für jeden Fall den gleichen Test aus. Wenn ein Test fehlschlägt, sieht man sofort am Namen, welches Szenario das Problem hat.

3.3. Integrationstests

Die Integrationstests sind das Herzstück der Teststrategie. Sie testen nicht einzelne Funktionen, sondern komplette User-Workflows. Dafür wurde ein eigenes Test-Framework entwickelt, mit der man UI-Interaktionen beschreiben kann.

Ein Test für einen Rebase sieht zum Beispiel so aus:

```

var Rebase = NewIntegrationTest(NewIntegrationTestArgs{
    Description: "Rebase onto another branch, deal with the conflicts.",
    SetupRepo: func(shell *Shell) {
        shared.MergeConflictsSetup(shell)
    },
    Run: func(t *TestDriver, keys config.KeybindingConfig) {
        t.Views().Commits().TopLines(
            Contains("first change"),
            Contains("original"),
        )

        t.Views().Branches().
            Focus().
            Lines(
                Contains("first-change-branch"),
                Contains("second-change-branch"),
            ).
    },
}

```

```

        SelectNextItem().
        Press(keys.Branches.RebaseBranch)

    t.ExpectPopup().Menu().
        Title(Equals("Rebase 'first-change-branch'")).
        Select(Contains("Simple rebase")).
        Confirm()

    t.Common().AcknowledgeConflicts()
},
}))

```

Der Test läuft in drei Schritten ab: Zuerst wird ein echtes Git-Repository mit dem Shell-Helper vorbereitet. Dann simuliert der TestDriver Benutzeraktionen wie `Focus()`, `SelectNextItem()` oder `Press()` – das entspricht den Tastendrücken, die ein echter Nutzer machen würde. Zwischendurch wird immer wieder geprüft, ob die UI das Erwartete anzeigt, zum Beispiel mit `Contains()` oder `Equals()`. Man kann den Test lesen wie eine Beschreibung dessen, was ein Nutzer tut.

Es gibt über 450 solcher Integrationstests, die alle möglichen Szenarien abdecken: Branch-Operationen wie Checkout oder Rebase, Commit-Operationen wie Amend oder Cherry-Pick, interaktive Rebases, Konfliktbehandlung, File-Operations und mehr.

Technisch basiert das Framework auf einer Architektur spezialisierter Driver-Komponenten. Der `ViewDriver` ermöglicht Interaktionen mit Listen-Views wie Branches, Commits und Files, während der `MenuDriver` die Navigation in Popup-Menüs steuert. Der `PromptDriver` behandelt Texteingaben, der `ConfirmationDriver` das Bestätigen oder Ablehnen von Dialogen und der `AlertDriver` die Validierung von Fehlermeldungen. Diese Abstraktion entkoppelt die Testlogik von der konkreten UI-Implementation, was Refactorings erheblich erleichtert. Ändert sich die Struktur der Benutzeroberfläche, müssen lediglich die Driver-Implementierungen angepasst werden, während die hunderten von Tests unverändert bleiben können.

4. Continuous Integration Pipeline

Die CI-Pipeline ist kein Bestandteil der Teststrategie selbst, sondern dient als technisches Mittel zur automatisierten Umsetzung der beschriebenen Testkonzepte. Lazygit nutzt GitHub Actions, um bei jedem Push und Pull Request automatisch die Qualitätssicherung durchzuführen.

Die Pipeline führt parallel mehrere Prüfungen aus: Unit-Tests laufen auf Ubuntu und Windows (Cross-Platform-Kompatibilität), Integrationstests validieren die Funktionalität gegen vier verschiedene Git-Versionen (2.32.0 bis neueste), Build-Jobs kompilieren für Linux, Windows und macOS, Konsistenz-Checks prüfen Dependencies und generierte Dateien, und golangci-lint führt statische Code-Analyse durch. Nach erfolgreicher Testausführung werden Coverage-Daten zu Codacy hochgeladen.

5. Entwurf eigener Testfälle

Der praktische Teil dieser Arbeit bestand darin, Testlücken im Lazygit-Projekt zu identifizieren und durch eigene Testfälle zu schließen. Dabei wurden die zuvor analysierten Teststrategien angewendet und die Coverage messbar verbessert.

5.1. Identifikation von Testlücken

Die Analyse der Coverage-Reports aus der CI-Pipeline in Kombination mit manuellen Code-Reviews identifizierte zwei signifikante Testlücken. Die Tag-Kommandos in `pkg/commands/git_commands/tag.go` enthielten acht Funktionen ohne jegliche Testabdeckung. Tags sind in Git zentral für die Versionsverwaltung, weshalb fehlerhafte Implementierungen zu versehentlich überschriebenen oder falsch gesetzten Tags führen können. Die zweite Lücke betraf die `StringStack`-Datenstruktur in `pkg/utils/string_stack.go`, eine LIFO-Implementierung ohne Tests. Die Priorisierung erfolgte nach Kritikalität, wobei die Tag-Funktionen als wichtiger eingestuft wurden.

5.2. Analyse und Testfalldesign

Die Tag-Funktionen konstruieren Git-Befehle programmatisch mit Hilfsfunktionen wie `NewGitCmd` und `ArgIf`, die Argumente nur unter bestimmten Bedingungen hinzufügen. Beispielsweise fügt `ArgIf(force, "--force")` das Force-Flag nur bei Bedarf hinzu. Diese bedingte Logik ist fehleranfällig und erfordert Tests für verschiedene Parameter-Kombinationen.

Für die Tag-Kommandos wurde Table-Driven Testing gewählt, da dies dem Projekt-Standard entspricht und sich optimal für Parametervariationen eignet. Die Tests für `CreateLightweightObj` decken vier Szenarien ab: einfacher Tag auf HEAD, Tag auf spezifischem Commit, Force-Flag zum Überschreiben existierender Tags und die Kombination aller Parameter. Für `CreateAnnotatedObj` wurden analoge Tests mit zusätzlichem Message-Parameter entwickelt. Die Funktion `IsTagAnnotated` parst Git-Ausgaben („commit“ vs. „tag“) und erhielt Tests mit verschiedenen Ausgabeformaten inklusive Whitespace-Varianten.

Für `StringStack` wurde ein zustandsbasierter Testansatz gewählt. Die Tests validieren LIFO-Semantik (`TestStringStack_PushAndPop`), das Verhalten bei leerem Stack (`TestStringStack_PopEmptyStack`), Zustandsprüfung (`TestStringStack_IsEmpty`), vollständigen

diges Zurücksetzen (`TestStringStack_Clear`) und komplexe Operationssequenzen (`TestStringStack_MultipleOperations`).

5.3. Implementierung

Die Tag-Tests wurden in `tag_test.go` implementiert und folgen strikt den Projekt-Konventionen. Jeder Testfall definiert eine Szenario-Struktur mit Eingabeparametern und erwarteten Git-Argumenten:

```
{
    testName: "create lightweight tag on HEAD",
    tagName: "v1.0",
    commitSha: "",
    force: false,
    expectedArgs: []string{"tag", "v1.0"},
}
```

Der `FakeCmdObjRunner` simuliert Git-Befehle ohne tatsächliche Ausführung. Für jeden Testfall werden die erwarteten Argumente beim Fake-Runner registriert, die Funktion ausgeführt und anschließend mit `CheckForMissingCalls()` validiert, dass alle erwarteten Befehle aufgerufen wurden.

Die `StringStack`-Tests verwenden klassisches Unit-Testing ohne Table-Driven-Ansatz, da primär Zustandsübergänge getestet werden:

```
func TestStringStack_PushAndPop(t *testing.T) {
    stack := NewStringStack()
    stack.Push("first")
    stack.Push("second")

    assert.Equal(t, "second", stack.Pop())
    assert.Equal(t, "first", stack.Pop())
}
```

5.4. Testergebnisse und Coverage-Verbesserung

Insgesamt wurden 18 Tests implementiert: 13 für Tag-Operationen und 5 für `StringStack`. Die lokale Ausführung mit `go test ./pkg/commands/git_commands -run TestTag -v` dauert unter 10 Millisekunden. Die CI-Pipeline validierte alle Tests erfolgreich auf Ubuntu und Windows ohne plattformspezifische Probleme.

Die Coverage-Analyse zeigt signifikante Verbesserungen für `tag.go`, wo fünf von acht Funktionen auf 100% Coverage gebracht wurden:

- NewTagCommands: 0% → 100%
- CreateLightweightObj: 0% → 100%
- CreateAnnotatedObj: 0% → 100%
- LocalDelete: 0% → 100%
- IsTagAnnotated: 0% → 100%

Die drei verbleibenden Funktionen (HasTag, Push, ShowAnnotationInfo) sind Remote-Operationen, die Netzwerk-Kommunikation erfordern und besser durch Integrationstests abgedeckt werden. Die Gesamt-Coverage von `tag.go` stieg von 0% auf 62.5%.

Für `string_stack.go` wurde vollständige Coverage erreicht:

- Push: 0% → 100%
- Pop: 0% → 100%
- IsEmpty: 0% → 100%
- Clear: 0% → 100%

Auf Package-Ebene verbesserte sich `pkg/commands/git_commands` von 37.1% auf 37.6% (+0.5 Prozentpunkte) und `pkg/utils` von 58.2% auf 59.6% (+1.4 Prozentpunkte). Obwohl die prozentualen Verbesserungen moderat erscheinen, schließen sie konkrete Lücken in wichtigen Funktionen innerhalb umfangreicher Packages.

6. Testauswertung und Metriken

Die Coverage-Verbesserungen sind messbar und signifikant. Für `tag.go` stieg die Coverage von 0% auf 62.5%, wobei alle getesteten Funktionen 100% Coverage erreichten. Nur drei Remote-Funktionen blieben ungetestet. `string_stack.go` erreichte vollständige 100% Coverage für alle Funktionen.

Auf Package-Ebene verbesserte sich `pkg/commands/git_commands` um 0.5 Prozentpunkte (37.1% → 37.6%) und `pkg/utils` um 1.4 Prozentpunkte (58.2% → 59.6%). Diese scheinbar kleinen Zahlen sind bedeutsam, da beide Packages umfangreich sind und die neuen Tests gezielt Lücken schließen.

Lazygit verfügt über 80+ Test-Dateien mit 2500 Unit-Test-Funktionen. Unit-Tests laufen in unter einer Minute. Die Test-Code-Ratio ist ausgewogen – kritische Packages haben umfangreichere Tests. Die Tests integrierten sich nahtlos in die CI-Pipeline durch Standard-Go-Patterns und laufen auf allen Plattformen.

7. Fazit

Quellen

- [1] E. W. Dijkstra, „The Humble Programmer“, *ACM Turing Award Lectures*. Association for Computing Machinery, New York, NY, USA, 2007. doi: 10.1145/1283920.1283927.
- [2] P. Liggesmeyer, *Software-Qualität: Testen, Analysieren und Verifizieren von Software*, 2. Auflage. Spektrum Akademischer Verlag, 2009.
- [3] D. W. Hoffmann, *Software-Qualität*, 2. Auflage. Springer Vieweg, 2013.
- [4] The Go Authors, „Table Driven Tests“. Zugegriffen: 15. Januar 2026. [Online]. Verfügbar unter: <https://github.com/golang/go/wiki/TableDrivenTests>